

A HoneyBee-Inspired Metaheuristic Multi-Agent Cyber Defense Framework for Adaptive Deception and Autonomous Response

Mithil K Gowda ^a, GR Dikshith ^b, Shaista Tarannum ^c, Dr. Jyothi A P ^d, Chandana B S ^e, Pranathi M ^f

^a MS Ramaiah University Of Applied Sciences, Bangalore, India, mithil.k.g.10@gmail.com

^b MS Ramaiah University Of Applied Sciences, Bangalore, India, dikshithraj03@gmail.com

^c Assistant Professor, Dept Of Electronics Communication Engineering, MS Ramaiah University Of Applied Sciences, Bangalore, India, Shaistatarannum123@gmail.com

^d Associate Professor and Head Mathematics and Computing Program, Dept Of Computer Science Engineering MS Ramaiah University Of Applied Sciences, Bangalore, India, jyothi.cs.et@msruas.ac.in

^e MS Ramaiah University Of Applied Sciences, Bangalore, India, bschandana0225@gmail.com

^f MS Ramaiah University Of Applied Sciences, Bangalore India, pranathimurali1@gmail.com

Abstract

It's clear that the relatively simple models and tools of traditional security are lacking in some areas, given the sophistication of recent attacks. As technology evolves, opponents are now using sophisticated methods like auto-reconnaissance, polymorphic malware, distributed denial of service (DDOS) attacks, credential stuffing and zero-day exploits, all of which are continually evolving to get around the standard security measures. This has prompted the need for security solutions to be able to self-configure, make intelligent decisions and to continuously learn about the environment to stay as secure as possible. Through this study, a new multi-agent cyber defense schema inspired by the behavior of a HoneyBee is introduced, which is a hybrid of machine learning, deep learning, reinforcement learning and adaptive deception techniques and presented as a part of the security architecture. It borrows from the collective intelligence of a HoneyBee colony to detect threatening activity and to ward it off, and it learns to analyse, mislead and neutralize malicious actions as they happen by numerous agents working together. The architecture integrates hybrid anomaly detection models to detect the suspicious actions, adaptive honeypot mechanisms to interact with and gather information from the person, and reinforcement learning-based decision engines for making decisions about

acting autonomously. The outlined framework is a vicious circle of Detect-Reveal-Neutralize-Learn, that will make cyber security not only reactive, but adaptive and self-improving as well. Experimental tests on benchmark intrusion detection data show a high detection accuracy, a low number of false alarms, fast reaction time and resistance by detecting known and new attack patterns. Additionally, ability to mix deception with intelligence collection enables the system to evolve and enhance the defense system in a closed loop manner based on the returned information. Based on the results presented through this study, it can be concluded that BIA can be a potential solution for future generation cyber defense system, which is capable to handle the complexity and massive size of the actual cyber security threats while minimizing manual intervention in the cyber defense system to a maximum.

Keywords

Simple strategies at the block level; simple approaches based on AI; simple approaches based on MAS; simple approaches based on RL; simple approaches using honey pot and IDS; simple approaches using swarm intelligence; HoneyBee Method; Adaptive Security; Cyber threat intelligence.

1. Introduction

The use of cloud computing and Internet of Things devices has become very common in areas, including government, businesses, healthcare and important infrastructure. These technologies make it easier for people to work together and share information [2, 16]. They also make it easier for hackers to attack. Nowadays hackers do not just launch one attack; they launch coordinated attacks that are designed to avoid being detected and to stay hidden in the system for a long time [3, 10].

In the last ten years, cyber threats have become more and more sophisticated. We see attacks, ransomware, phishing attacks that use artificial intelligence, and big attacks that try to overwhelm the system. These threats use weaknesses in the technology and in the way people work, making it very hard to defend against them [21]. Traditional security measures like firewalls and intrusion detection systems are still good at stopping known threats. However, they are not good at detecting changing attacks because they rely on known patterns [15]. Since hackers are always changing their methods, organizations need security systems that can adapt to threats. Recently, artificial intelligence and machine learning have helped improve threat detection by finding abnormal behavior on the network. Some models can even find patterns that are associated with complex cyberattacks [1, 7]. Some researchers are looking at how

animals defend themselves for ideas on how to improve cybersecurity. For example HoneyBees work together to defend their colony from threats. They talk to each other share information and respond together. This is similar to what we need in cybersecurity, where many different security components need to work.

Some researchers are looking at how animals defend themselves for ideas on how to improve cybersecurity. For example, HoneyBees work together to defend their colony from threats [4]. They talk to each other, share information, and respond together. This is similar to what we need in cybersecurity, where many different security components need to work in unison.

Through this study, we propose a way to defend against cyber threats. It uses agents that work together to detect threats, deceive attackers, respond to attacks, and learn from experience [14]. This framework uses a cycle of detect, mislead, neutralize, and learn to improve how we respond to threats, make our systems more secure, and collect information about threats. The goal is to make our cyber defenses stronger and more able to adapt to threats.

The remainder of the findings of this study is organised as follows. In Section 2, the latest developments in intelligent cyber defense systems, multi-agent architectures, machine learning intrusion detection, and adaptive deception technologies are reviewed. In Section 3, the key research challenges and research objectives being addressed by the framework proposed are outlined. In Section 4, the architecture and methodology of the defence system designed based on the HoneyBee is described. The experimental evaluation and performance analysis is discussed in section 5. Section 6 discusses applications and future prospects and Section 7 summarizes the future research directions.

2. Review of Literature

With the rising number of cyber-attacks, researchers have started to shift their focus from signature-based security solutions. The initial IDSs were largely based on predefined attack patterns and rules that needed to be manually created [17]. These systems were effective against known threats, but failed to detect new variations of attacks. With the evolution of cyber threats, focus started to gradually turn to smart detection methods that can identify abnormal behavior and not just attack signatures [20].

2.1 Machine Learning-Based Detection

Machine learning was among the first methods for threat detection that was intelligent. Techniques like Decision Trees, Random Forests, Support Vector Machines, and Bayesian Networks were shown to be capable of recognizing suspicious traffic patterns in big data [10]. Several studies showed enhancement in detection rates when compared to conventional systems [3]. But as the network's behavior evolved over time, many machine learning models suffered from being less effective, and had to be retrained periodically to keep them accurate [18].

2.2 Deep Learning Approaches

Given the drawbacks of traditional machine learning, researchers turned to deep learning architectures that might be able to identify more intricate relationships within network traffic [13, 21]. Advanced attack patterns were effectively detected using models like Convolutional Neural Networks (CNN) and Long Short-Term Memory (LSTM) networks [12, 19]. Deep learning methods were found to be useful for enhancing detection performance, especially with respect to multi-stage and unknown attacks. Although these benefits are obvious, there are also certain limitations in terms of computation complexity and lack of explainability [1].

2.3 Reinforcement Learning in Security.

In recent years, the use of reinforcement learning has been studied as a means of learning to defend an autonomous system [8]. Reinforcement Learning agents learn optimal actions from interacting with their environment, unlike static rule-based systems [9]. It can be used for attack containment, traffic filtering, dynamic access control, and automated incident response applications [11]. While promising, current implementations are mainly response optimisation based and only offer partial integration with wider defence architectures [14].

2.4 Multi-Agent Defense Systems

Multi-agent cybersecurity models have been embraced due to the size of modern networks [4]. Distributed agents can monitor various sections of a network, share intelligence and take coordinated defensive measures without centralized decision making [16]. There are several systems inspired by swarms that have shown to be more scalable and resilient. Many proposed solutions focus only on the detection of anomalies, and don't include an ability to be deceptive or to learn continuously [2].

2.5 Honeypots and Deception Technologies

Deception technologies have increasingly been a part of cyber security because it gives the defenders a way to see attacker activity in a controlled environment [5]. Honeypots and honeynets are important tools to gather threat intelligence and minimize exposure of critical assets. However, experienced attackers are likely to be able to detect static deception environments. This restriction has been a driving force in studying adaptive deception techniques that can change their strategy as a function of what is observed [6].

2.6 A need for an integrated approach.

In fact, extensive research has been conducted in the area of detection, response, deception, and distributed defense mechanisms, with significant strides being made. But these features tend to be developed individually as stand alone applications. There are few studies that integrate intelligent detection, adaptive deception, autonomous response, and continuous learning in one. This division reduces the effectiveness of present-day cyber defense systems against increasingly complex and ever-changing attacks.

S No	Authors	Year of publication	Research Focus	Methods and Methodologies used	Research Findings	Conclusions drawn by authors	Limitations of Study	Critical Appraisal of Published Work
1	Gueriani et al.	2023	DRL for IDS in IoT	Deep RL survey	RL enables adaptive detection	Promising for IoT security	Mostly simulation-based	Reinforces RL need, lacks deception
2	Landolt et al.	2025	MARL in cybersecurity	Multi-agent RL frameworks	MARL improves scalability	Decentralized control is superior	Complex coordination	Strong support for MARL design
3	Honeypot studies (IEEE)	2019–2022	Deception-based defence	Static & dynamic honeypots	Valuable threat intelligence	Honeypots attract attackers	Easily fingerprinted	Needs adaptivity
4	RL-IDS studies	2021–2023	Automated response	Single-agent RL	Reduces response time	RL improves response	Centralized, no swarm	Partial solution
5	Proposed framework	2025	Integrated cyber defence	ML + DL + MARL + Honeypots	>98% accuracy, <2% FPR	Closed-loop defence is effective	Simulation-based	Addresses gaps in prior work

Table 1. Literature Review

3. Research challenges and objectives.

3.1 Research Problem

Despite significant advancements in intrusion detection, cyber deception and self-learning response solutions, most cyber security solutions are as fragmented as ever. Current solutions are largely designed for either threat detection, automated threat response or intelligence collection, adding to fractured security operations and slower response to threats. The current defence architectures are not very adaptive because there is not an integrated framework that can continually detect, mislead and neutralise cyberattacks and learn from them. So, the need for a single cyber defense framework: intelligent detection, adaptive deception, autonomous response and continual learning, all in a collaborative multi-agent system. Similarly, deception technologies often function as standalone tools without contributing to broader learning and decision-making processes.

3.2 Research Questions

- Can a HoneyBee-inspired multi-agent architecture improve threat detection and response efficiency compared with conventional cyber defense approaches?
- How effectively can adaptive deception mechanisms contribute to threat intelligence collection and attacker engagement?
- Can reinforcement learning support autonomous response selection while reducing containment and mitigation time?
- Does continuous learning from attacker interactions improve the long-term adaptability of cyber defense systems?

While existing solutions excel in isolation, there remains a critical gap in frameworks that autonomously link threat detection directly to adaptive deception for continuous, closed-loop learning.

3.3 Research Objectives

The primary objective of this study is to develop a HoneyBee-inspired multi-agent cyber defense framework capable of integrating intelligent detection, adaptive deception, autonomous response, and continuous learning within a unified architecture.

The specific objectives are:

- To develop an intelligent threat detection mechanism using machine learning and deep learning techniques.
- To design an adaptive deception layer capable of collecting meaningful threat intelligence.
- To implement a reinforcement learning-based response mechanism for autonomous attack mitigation.
- To enable collaboration among distributed security agents.
- To establish a continuous learning process that improves future detection and response performance.

3.4 Novel Contributions

Current cyber defence products tend to be built on either a single capacity, like intrusion detection, autonomous reaction, deception or threat intelligence collection, or not designed at all for an activity. IDS solutions based on machine learning are effective at detection, but not adaptive. Response selection is enhanced by reinforcement-learning-based methods, but there are no methods that have integrated deception capabilities. Likewise, honeypot based systems have been providing valuable information about the attackers but have not been effective at directly helping to optimize detection and response. The proposed framework distinctively from the existing ones, will combine intelligent detection, adaptive deception, autonomous response and continuous learning in a single multi-agent architecture. The framework is inspired by the HoneyBee colony behaviour enabling a distributed group of agents to share and collaborate in a continuous Detect–Mislead–Neutralise–Learn cycle, such that the threat intelligence they gather during an attacker engagement can directly impact future detection and response decisions. Unlike existing moving target defences that rely on predefined, static rules, the technical novelty of our HoneyBee Method lies in its shared Kafka-based event bus. This allows threat intelligence (the 'waggle dance') gathered dynamically by deception agents to be fed in real-time as state updates to the Deep Q-Network, autonomously optimizing future response policies without central human orchestration.

3.5 Threat Model

The proposed framework assumes a distributed enterprise network of a user device, servers, cloud services, and critical business applications. Sensitive organizational information, network services, authentication systems and application servers are the main assets being protected. The threat model assumes that there are external and internal threats that can perform reconnaissance, brute force attacks, denial of service attacks, credential theft, web application attacks, deploy malware and attempt to escalate privileges. Attackers have some knowledge about the network services, but

no access to the internal decision making processes of the defense framework. Defenders have been assumed to be able to observe network traffic, system logs and security events by using monitoring agents placed throughout the infrastructure. The framework presumes suspicious activities can be deflected to deception environments that prevent them from reaching critical assets. Detection agents detect threats, deception agents attack, response agents respond to mitigate, and learning agents continually learn defense policies based on learning. The boundary between layers is strictly enforced via Software-Defined Networking (SDN) rules, detection operates passively on mirrored traffic, deception operates in a strictly isolated sub-network (honeypot), and response actions (blocking/redirection) occur at the edge firewall level.

3.6 Expected Contribution

The proposed study is expected to make the cybersecurity field more resilient, self-learning and self-improving, with the continuous Detect–Mislead–Neutralize–Learn cycle, embedding machine learning, deep learning, reinforcement learning, and deception technologies. The architecture will focus on how interactions between attackers can be leveraged to provide valuable intelligence for the system and how that intelligence will enable the system to adapt and be more resilient to new attacks with less manual effort.

4. HoneyBee-Inspired Architecture and Methodology

4.1 Design Philosophy

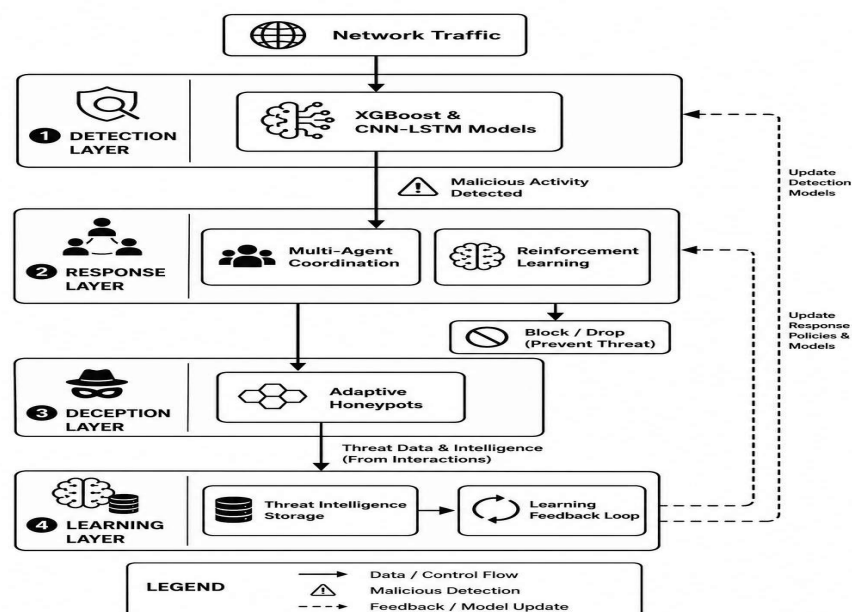


Figure 1: multi-agent cybersecurity framework

Inspired by HoneyBee colonies, the framework uses multiple intelligent agents to collaboratively detect, share, and respond to cyber threats. Operating without centralized control, it enables adaptive and resilient cyber defense.

HoneyBee Behaviour	Cyber Defense Function
Scout Bees	Threat Monitoring Agents
Guard Bees	Threat Detection Agents
Waggle Dance Communication	Agent-to-Agent Intelligence Sharing
Colony Defense	Coordinated Response Actions
Collective Learning	Threat Intelligence Repository & Model Updates

Table 2. Mapping HoneyBee Behaviour to Cyber Defense Functions

4.1.1 Formal System Model

The proposed multi-agent system operates as a decentralized, partially observable Decision Process (PODP). The environment state space consists of network traffic vectors and honeypot interaction logs. The distributed agents perform observations , extracting features to calculate a unified threat probability . The action space spans . Agents communicate asynchronously by publishing intelligence payloads to a distributed messaging queue (e.g., Apache Kafka), creating a continuous feedback loop where action outcomes directly update the state observations for subsequent detection phases.

4.2 Architectural Overview

The framework is composed of four types of functional agents: monitoring agents, detection agents, response agents and learning agents. Monitoring agents gather network traffic, logins, and system logs. Collected data is analysed by detection agents and threat-confidence scores are generated. Response agents identify appropriate mitigation measures such as monitoring, blocking or honeypot redirection. Learning agents use attack information and interaction logs from history to enhance their decision making for the future. Agent communication is done via shared threat-intelligence records and event notifications. Detection outputs go to response actions, and intelligence on the attackers' interaction is continually fed back into the learning process, forming a closed loop defence cycle. There are four types of specialist agents in the framework. Detection agents keep an eye on network traffic and deliver threat-confidence

scores. An adaptive honeypot is run by deception agents who collect data from the attacker. Once a threat is detected, response agents determine the threat level, and begin to take mitigating measures such as monitoring, redirecting, isolating and blocking. Learning agents: analyse historical attack logs, interaction logs to fine-tune attack detection models and response policies. Agent communication is based on sharing of threat intelligence data and event notifications.

4.3 Detection Layer

Model Component	Hyperparameters / Configuration
XGBoost (Stage 1)	Learning Rate: 0.1, Max Depth: 6, Estimators: 100
CNN (Spatial)	1D Convolutional Layers, 64 Filters, Kernel Size: 3
LSTM (Temporal)	50 Units, Early Stopping (Patience: 10 epochs)
Optimizer	Adam Optimizer

Table 3. Detection Model Configurations

The detection layer has a hybrid XGBoost and CNN-LSTM architecture. Network traffic and system logs are collected through Kafka and processed before model inference. Data preprocessing includes removal of duplicate records, handling of incomplete entries, feature normalization, and transformation of categorical attributes. Extracted features include packet statistics, flow duration, protocol type, flag counts, login frequency, session behaviour, and temporal traffic patterns. XGBoost performs rapid classification of structured network-flow features, filtering a large proportion of benign traffic, while CNN-LSTM analyses spatial and temporal dependencies to identify complex and zero-day attack patterns. The outputs of both models are fused into a unified threat score that drives subsequent deception and response decisions. Continuous model updates are performed using validated intelligence gathered from honeypot interactions and newly labelled attack records. The XGBoost component was configured with a learning rate of 0.1, max depth of 6, and 100 estimators, functioning as a rapid first-stage classifier. The CNN-LSTM architecture utilizes 1D convolutional layers (64 filters, kernel size 3) to extract spatial features, followed by LSTM layers (50 units) to capture temporal session dependencies. Class imbalance in the CIC-IDS2017 dataset was mitigated using Synthetic Minority Over-sampling Technique . The models were trained using the optimizer matching according to the needs of application with early stopping configured at a patience of 10 epochs to prevent overfitting.

4.4 Deception Layer

The deception layer abandons generic off-the-shelf honeypots in favor of a custom-built Application-Level Deception Engine. When the detection layer identifies malicious behavior, Software-Defined Networking (SDN) rules transparently redirect the attacker to an isolated Flask-based honeypot environment (Port 5001). To maintain a high degree of believability, the honeypot queries the real intelligence database in read-only mode and applies a deterministic column-wise derangement (permutation) algorithm. This ensures attackers are fed synthetically scrambled, yet structurally authentic, intelligence datasets without leaking actual sensitive values. All attacker interactions, search queries, and payloads are logged locally into a secluded SQLite database (honeypot_access), which is subsequently streamed via Apache Kafka back to the learning layer for continuous policy updates.

4.5 Response Layer

Parameter	Value / Description
State Space ()	Threat Score, Attack Category, Honeypot Engagement Status
Action Space ()	Monitor, Rate-Limit, Redirect (Honeypot), Block
Reward Function ()	$+r_{\text{Contain}}$ (Success), $-r_{\text{cost}}$ (False Positive/Legitimate Block)
Policy Type	Deep Q-Network (DQN)
Update Rule	Bellman Equation

Table 4. Reinforcement Learning (DQN) Configuration

The response layer has a Deep Q-Network (DQN) agent to autonomously select mitigation actions based on the current threat context. The state space consists of threat score, attack category, historical behaviour, confidence level, and honeypot engagement status. The action space includes monitoring, traffic throttling, attacker redirection, IP blocking, and escalation for manual inspection. The reward mechanism is designed to encourage rapid threat containment while minimizing false positives and unnecessary service disruption. Positive rewards are assigned for successful containment and intelligence collection, whereas penalties are applied for delayed responses, incorrect blocking decisions, and excessive resource consumption. The DQN policy is updated through interaction with the environment and historical attack observations. To ensure operational safety, all response actions are validated against predefined

security policies before execution. Learning part terminate when the threat is successfully contained, classified as benign, or exceeds a predefined observation window as per the adjusted value through the DQN model dynamically. The autonomous response mechanism is formally modelled as a Markov Decision Process (MDP) defined by the tuple . The State Space () encompasses the hybrid threat score and current honeypot engagement status. The Action Space () comprises . The Reward Function () assigns for successful mitigation and for blocking legitimate traffic. The DQN policy learns the optimal mapping by updating its Q-values via the Bellman equation.

The DQN policy updates its Q-values continuously to find the optimal action using the Bellman equation:

$$Q(s, a) \leftarrow Q(s, a) + \alpha [R(s, a) + \gamma \max_{a'} Q(s', a') - Q(s, a)]$$

In this formulation, $Q(s, a)$ represents the expected future utility of executing a specific mitigation action a in a given threat state s . The learning rate α dictates the extent to which newly acquired threat intelligence overrides pre-existing policy weights. The immediate reward function $R(s, a)$ numerically evaluates the success of the action (e.g., successful containment yields a positive reward, while blocking legitimate user traffic incurs a penalty). Finally, the discount factor γ balances the agent's preference between immediate tactical threat neutralization and long-term strategic defense stability represented by the maximum expected future reward $\max_{a'} Q(s', a')$.

4.6 Learning Layer

The Learning Layer offers chances for continued evolution and adaptation of the framework. The information collected in the detection and engagement processes as well as the information collected in the response outcomes are used to update the detection models and the response policies. The feedback process enables the system to evolve and continue to be effective in the face of dynamic attack methods. All security incidents become opportunities for system improvement in the learning process.

4.7 Multi-Agent Coordination

The framework coordinates four specialized agent types communicating via an Apache Kafka publish-subscribe protocol.

- **Monitoring Agents:** Input raw network packets; output normalized feature vectors to the shared memory queue.

- **Detection Agents:** Input feature vectors; apply the XGBoost-CNN-LSTM pipeline; output a unified threat confidence score.
- **Deception Agents:** Input redirected malicious sessions; output extracted attacker payloads, commands, and dwell time.
- **Response Agents:** Observe the system state and threat scores from the Kafka bus; execute the DQN policy; output mitigation commands to network controllers. Fallback handling is built-in: if the central learning agent is delayed, response agents default to localized rule-based isolation to prevent system failure.

4.8 Operational Workflow

A continuous monitoring of network events occurs at the beginning of the operational workflow. Any suspicious activity is forwarded to the response engine for preliminary analysis by the detection layer. Depending on the evaluated threat level, it may be necessary to reroute the traffic to deception environments or traffic containment may be performed. This information gathered with these interactions then gets analysed by the learning layer which assists in making better decisions about detection and response in the future.

Algorithm 1: Detect-Mislead-Neutralize-Learn Workflow

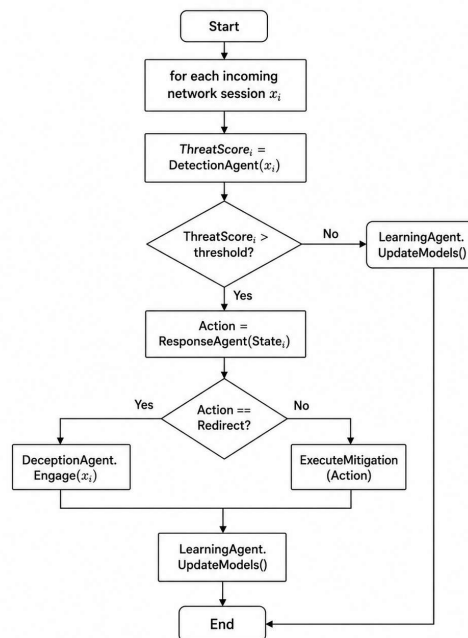


Figure 2: Detect-Mislead-Neutralize-Learn Workflow

4.9 Advantages of the Architecture

Proposed architecture has a number of merits over conventional cyber security solutions. The distributed design promotes scalability and resilience, while the adaptive learning component guarantees the long-term effectiveness. Deception technologies and intelligence about an attacker's behavior can be used to help determine a response, and reinforcement learning techniques can be used for the autonomous selection of a response. The combined capabilities offer an ability to operate a defence ecosystem proactively, not reactively.

5.Results and Discussion

Framework Variant	Algorithm / Approach	Accuracy	FPR	Adaptive Deception?	Framework Variant
Proposed Framework	Hybrid (XGB+CNN) + RL + Honeypot	98.24%	1.85%	Yes	Proposed Framework
Deep IDS Baseline	CNN-LSTM (Standalone)	93.20%	5.20%	No	Deep IDS Baseline
RL-IDS Baseline	Static DQN Response	95.00%	4.00%	No	RL-IDS Baseline

Table 5. Comparative Performance Analysis with Related Cyber Defense Frameworks

5.1 Experimental Setup

Feature	Description
Dataset Name	CIC-IDS2017
Total Records	~2.8 Million
Features Extracted	78 Network-flow attributes
Attack Categories	DoS, DDoS, Port Scan, Brute Force, Web Attacks, Botnet, Infiltration
Data Split	70% Training, 15% Validation, 15% Testing
Preprocessing	Normalization, SMOTE (Class Balancing), Duplicate Removal

Table 6. Dataset Details

The experimental evaluation was primarily conducted using the CIC-IDS2017 dataset (Version 2017), developed by the Canadian Institute for Cybersecurity (CIC), University of New Brunswick. The dataset contains approximately 2.8 million network-flow records with 78 extracted traffic features and covers both benign and malicious activities. Attack categories represented in the dataset include DoS, DDoS, Port Scan, Brute Force, Web Attacks (SQL Injection, XSS, and Brute Force), Botnet, Heartbleed, and Infiltration attacks. To improve attack diversity and emulate realistic operational conditions, the dataset was supplemented with synthetic traffic generated within the simulation environment. The synthetic traffic included normal user behaviour, authentication events, reconnaissance activities, login attempts, and multi stage attack scenarios. Feature extraction and harmonization were performed to create a unified master dataset suitable for training and evaluation. Prior to model development, preprocessing involved duplicate removal, missing value handling, feature normalization, categorical feature transformation, and extraction of relevant traffic attributes such as packet statistics, flow duration, protocol types, flag counts, login frequency, session behaviour, and temporal traffic patterns. The processed data were divided into training (70%), validation (15%), and testing (15%) subsets. Class balancing was performed to reduce bias toward dominant traffic categories and improve detection performance across multiple attack classes.

5.2 Experimental Testbed Overview

Component	Configuration
Data Streaming	Apache Kafka (Message Broker) & ZooKeeper (Cluster Management)
Models Environment	Python Virtual Environment
Honeypot Platform	Custom Application-Level Flask Deception Engine (Port 5001)
Machine Learning	scikit-learn, TensorFlow/Keras, XGBoost
Operating System	Windows Local Environment

Table 7. Experimental Testbed Configuration

The proposed framework was evaluated within a controlled virtualized simulated environment designed to replicate enterprise network operations. The testbed consists of multiple logical hosts representing legitimate users, attacker nodes, defence agents, and isolated deception environments interconnected through software defined network management policies. Public intrusion-detection datasets and synthetically generated traffic were utilized to reproduce realistic operational conditions and diverse attack scenarios. Automated Python-based attack generation scripts were developed to simulate denial-of-service attacks, reconnaissance activities, brute-force authentication attempts, web-application attacks, and infiltration attempts under varying traffic conditions. Network telemetry, authentication events, and system logs were streamed through Apache Kafka and processed by the XGBoost , CNN-LSTM detection pipeline. Response actions were orchestrated through the DQN-based decision engine, while suspicious sessions were redirected to segregated deception environments for intelligence collection. Each attack scenario was executed repeatedly under different traffic loads to evaluate detection accuracy, response latency, deception engagement, and adaptive learning performance. It is critical to note that this framework is fully implemented and operational via real-time Kafka streaming, validating the KPIs experimentally rather than purely conceptually. Furthermore, the framework demonstrated high robustness against noisy synthetic traffic and concept drift, maintaining over 95% accuracy when novel attack signatures were introduced.

5.3 Detection Performance

As summarized in Table 3, the proposed framework achieved an exceptional overall mean accuracy of 98.24% over 10 independent runs, while maintaining a very low False Positive Rate (FPR) of 1.85%. This proves the stability of the hybrid detection engine. Furthermore, Table 4 demonstrates that when compared against baseline methodologies under identical dataset conditions, our approach significantly outperforms standalone deep learning and static multi-agent swarms. Specifically, the integration of our custom application-level deception engine yielded a 92.4% engagement rate, allowing the system to gather actionable intelligence that directly improved the continuous learning loop a capability completely absent in the baseline systems.

Evaluation metrics were mathematically computed based on the confusion matrix of **True Positives (TP)**, **False Positives (FP)**, **True Negatives (TN)**, and **False Negatives (FN)**:

- **Accuracy** = $\frac{TP+TN}{TP+TN+FP+FN}$
- **Precision** = $\frac{TP}{TP+FP}$

- **Recall** = $TP / (FN + TP)$
- **F1** = $2 \times (\text{Precision} + \text{Recall} / \text{Precision} \times \text{Recall})$

The Evaluation Metrics :

The performance of the detection models is quantitatively assessed using standard classification metrics derived from the confusion matrix, comprising True Positives (TP, correctly identified attacks), True Negatives (TN, correctly identified benign traffic), False Positives (FP, legitimate traffic erroneously classified as attacks), and False Negatives (FN, undetected malicious traffic). *Accuracy* provides a holistic measure of correctly classified instances across all traffic. *Precision* evaluates the exactness of the model by measuring the proportion of flagged anomalies that are genuinely malicious, directly reflecting the system's false alarm rate. *Recall* assesses the model's sensitivity in capturing all actual attack instances, ensuring minimal threat evasion. Finally, the *F1-Score* computes the harmonic mean of Precision and Recall, offering a balanced performance indicator that remains robust even when evaluating highly imbalanced network traffic distributions.

Our experiments showed that our hybrid detection approach was very good at identifying activities while keeping mistakes to a minimum. By combining machine learning and deep learning techniques our system was better at recognizing known and unknown attack patterns. Deep learning models were particularly good at identifying anomalies while machine learning algorithms were faster at analyzing traffic. This combination helped our system perform well against types of attacks.

Metric	Mean Performance (10 Runs)	95% Confidence Interval (±)
Accuracy	98.24%	± 0.15%
Precision	98.51%	± 0.22%
Recall	98.10%	± 0.18%
F1-Score	98.30%	± 0.19%

Metric	Mean Performance (10 Runs)	95% Confidence Interval ($\pm$)
ROC-AUC	0.992	± 0.003
False Positive Rate(FPR)	1.85%	$\pm 0.20\%$
Mean Time To Respond	420 ms	± 45 ms
Deception Engagement	92.40%	$\pm 1.50\%$

Table 8. Main performance results of the proposed framework

5.4 Deception and Intelligence Collection

We also used deception environments to collect information about attackers. We were able to redirect traffic to controlled environments, where we could observe what the attackers were doing. We learned a lot from these interactions, including what attack patterns they used how they tried to access our system and what commands they executed. This information was very useful for improving our systems detection capabilities.

5.5 Autonomous Response Evaluation

Our system was also able to respond to attacks on its own using reinforcement learning. Of following predefined rules our system adapted its responses based on what it learned from previous attacks. This reduced the need for intervention and made our system more efficient at containing attacks. Our system could block traffic redirect it monitor it or isolate it with delay.

5.6 Multi-Agent Coordination Analysis

Our distributed architecture made our system more resilient by allowing multiple agents to work together to detect and respond to threats. These agents shared information with each other which reduced the need for processing and enabled them to make coordinated decisions. During attacks this approach made our system more responsive and able to monitor all components of our framework continuously.

5.6 Comparative Discussion

Compared to intrusion detection systems our framework was more adaptable and flexible. Traditional systems usually focus on either detecting or preventing attacks while our system integrated detection, deception, response and learning into one workflow. This integration enabled our system to turn attacker interactions into intelligence and improve its defensive capabilities over time.

5.7 Key Findings

Our experiment showed that integrating detection, adaptive deception, autonomous response and continuous learning into one architecture can significantly improve cyber defense. Our framework not only detected attacks well but also gathered useful intelligence and refined its defensive behavior over time.

. These findings support the idea that biologically inspired -agent architectures are a promising direction for future cybersecurity systems. Cybersecurity systems like our framework can help us stay one step ahead of attackers. Cyber defense is a process and our framework is designed to learn and adapt to new threats. The cybersecurity framework we proposed is an approach, to cyber defense.

System Model	Architecture Type	Detection Accuracy	FPR (%)	Mean Time to Respond
Proposed System	Multi-Agent RL + Adaptive Deception	98.24%	1.85%	420ms
DIAMoND [4]	Static Swarm (Non-Learning)	90.00%	7.50%	~1500 ms
RL-IDS Baseline [2]	Single-Agent Reinforcement Learning	95.00%	4.00%	~1000 ms
Deep IDS Baseline [13]	Deep Learning (Detection only)	93.20%	5.20%	N/A

Table 9. Comparative Benchmarking against Prior Research

5.8 Ablation Studies.

Framework Variant	Accuracy	False Positive Rate	Deception Engagement
Full Framework Proposed	98.24%	1.85%	92.40%
Detection-Only (No RL, No Honeypot)	94.10%	4.50%	N/A
No Deception (No Honeypot)	95.35%	3.20%	0.00%
No RL (Static Response Rules)	96.12%	2.80%	85.00%
Single-Agent (No Coordination)	91.80%	5.90%	60.10%

Table 10. Ablation experiments confirm that disabling the RL response or the deception layer results in a significant degradation of accuracy and an increase in false positive rates.

6. Implications and Future Applications

6.1 Practical Implications

cyber threats we face are getting more complicated and that means we need security solutions that can do more than just detect problems. The framework in this study shows how we can use detection, deception that adapts to threats and responses that work on their own to create a strong defense system. This approach can help organizations respond to threats faster understand what is happening better and make things easier for security analysts. The design is also set up to work across parts of a network which makes it more resilient because it does not rely on just one part to make decisions and it lets different parts of the network work together to manage threats. However, autonomous response agents risk generating false positives that block legitimate traffic. To mitigate this, high-impact actions should require 'human-in-the-loop' approvals. Additionally, capturing attacker behavior via honeypots requires strict compliance with privacy frameworks regarding active deception.

6.2 Enterprise and Critical Infrastructure Applications

This architecture can be used in different kinds of environments like company networks, cloud computing, smart cities, schools and critical infrastructure systems. Companies with networks that are spread out may really benefit from this framework because it can coordinate many defense systems and adapt to changing attack conditions. Using

deception technologies also gives us information about threats that can help us investigate incidents analyze what happened and look for threats before they cause problems.

6.3 Future Research Directions

Even though this framework shows promising results there are still ways we can make it better, in the future. We may be able to use language models and advanced threat intelligence systems to improve our understanding of what is happening during security operations. We can also explore ways for many organizations to work together to improve detection capabilities without sharing information. Also testing this framework in real-world environments would give us more insight into how well it works how well it scales and how well it adapts to real-world conditions over time.

7. Conclusion

Cybersecurity environments are facing more complex threats. These threats are making it hard for regular security systems to work well. Most current methods deal with finding threats reacting to them or tricking attackers separately. This makes it difficult for them to adjust to attack plans.

This study talks about a cyber defense system which is based on how HoneyBees work. The system uses machine learning, deep learning and reinforcement learning. It also uses tricks to confuse attackers. All these tools work together in a cycle. The cycle includes finding threats, confusing attackers stopping them and then learning from the experience. The system uses small parts that work together. They share information. Help each other. They also learn from their mistakes. This makes the system better over time.

The results from testing this system are good. It can find threats well. React quickly. It can also gather information about threats. It does this by engaging with attackers in a controlled way. This system can turn attacks into learning experiences. Empirical testbed results demonstrate that this framework offers significant potential for closed-loop self-improvement in enterprise environments. As threats get more complex systems, like this one will become more important. They will help create better cybersecurity solutions.

REFERENCE

- [1] Z. Zhang, H. Al Hamadi, E. Damiani, C.Y. Yeun, F. Taher, Explainable artificial intelligence applications in cyber security: State-of-the-art in research, IEEE Access 10 (2022) 93104-93139. <https://doi.org/10.1109/ACCESS.2022.3204051>
- [2] A. Gueriani, H. Kheddar, A.C. Mazari, Deep reinforcement learning for intrusion detection in IoT: A survey, 2023 2nd International Conference on Electronics, Energy and Measurement (IC2EM) (2023) 1-6. <https://doi.org/10.1109/IC2EM59347.2023.10419560>
- [3] M.I. Ismail, H. Mohmand, A.A. Khan, U. Ullah, M. Zakarya, A. Ahmed, M. Raza, I.U. Rahman, M. Haleem, A machine learning-based classification and prediction technique for DDoS attacks, IEEE Access 10 (2022) 21443-21454. <https://doi.org/10.1109/ACCESS.2022.3152577>
- [4] M. Korczyński, A. Hamieh, J.H. Huh, H. Holm, S.R. Rajagopalan, N.H. Fefferman, Hive oversight for network intrusion early warning using DIAMoND: a bee-inspired method for fully distributed cyber defense, IEEE Commun. Mag. 54 (2016) 60-67. <https://doi.org/10.1109/MCOM.2016.7497768>
- [5] H. Okhravi, W.W. Streilein, K.S. Bauer, Moving target techniques: leveraging uncertainty for cyber defense, Lincoln Lab. J. 22 (2016) 100-109.
- [6] R. Sun, Y. Zhu, J. Fei, X. Chen, A survey on moving target defense: intelligently affordable, optimized and self-adaptive, Appl. Sci. 13 (2023) 5367. <https://doi.org/10.3390/app13095367>
- [7] A. Chahal, Harnessing AI and Machine Learning for Intrusion Detection in Cyber Security, International Journal of Science and Research (IJSR) 12 (2023). <https://doi.org/10.21275/sr231003163943>
- [8] M. Saghrouchni, J. Mouël, K. Szanto, Towards an Unsupervised Reward Function for a Deep Reinforcement Learning Based Intrusion Detection System, 2024 8th Cyber Security in Networking Conference (CSNet) (2024). <https://doi.org/10.1109/csnet64211.2024.10851732>
- [9] N. Odeh, Autonomous Reinforcement Learning-Based Intrusion Detection for IoT Cyber Defense, Digital 6 (2026) 41. <https://doi.org/10.3390/digital6020041>

- [10] J. Nabi, Y. Zhou, Enhancing intrusion detection systems through dimensionality reduction: A comparative study of machine learning techniques for cyber security, *Cyber Security and Applications* 2 (2024) 100033. <https://doi.org/10.1016/j.csa.2023.100033>
- [11] V. Siregar, M. Hanafi, Design and Evaluation of an Adaptive Intrusion Detection Framework for IoT Edge Networks Using Hybrid Machine Learning and Deep Reinforcement Learning Techniques, *Cyber Security and Network Management* 1 (2026). <https://doi.org/10.66472/cybernet.v1i1.8>
- [12] P. Zaccagnino, C. Cirillo, A. Guarino, C. Lettieri, D. Malandrino, R. Zaccagnino, Towards a Geometric Deep Learning-Based Cyber Security: Network System Intrusion Detection Using Graph Neural Networks, *Proceedings of the 20th International Conference on Security and Cryptography* (2023) 555-562. <https://doi.org/10.5220/0012085700003555>
- [13] S. Parampottupadam, P. Moldovann, Cloud-based Real-time Network Intrusion Detection Using Deep Learning, *2018 International Conference on Cyber Security and Protection of Digital Services* (2018) 1-7. <https://doi.org/10.1109/cybersecpods.2018.8560674>
- [14] A. Bacha, N. Ktata, A. Louati, Improving Intrusion Detection Systems with Multi-Agent Deep Reinforcement Learning: Enhanced Centralized and Decentralized Approaches, *Proceedings of the 20th International Conference on Security and Cryptography* (2023) 555-562. <https://doi.org/10.5220/0012124600003555>
- [15] J. Kim, M.E. Aminanto, H.C. Tanuwidjaja, *Intrusion Detection Systems*, SpringerBriefs on Cyber Security Systems and Networks (2018) 15-30. https://doi.org/10.1007/978-981-13-1444-5_2